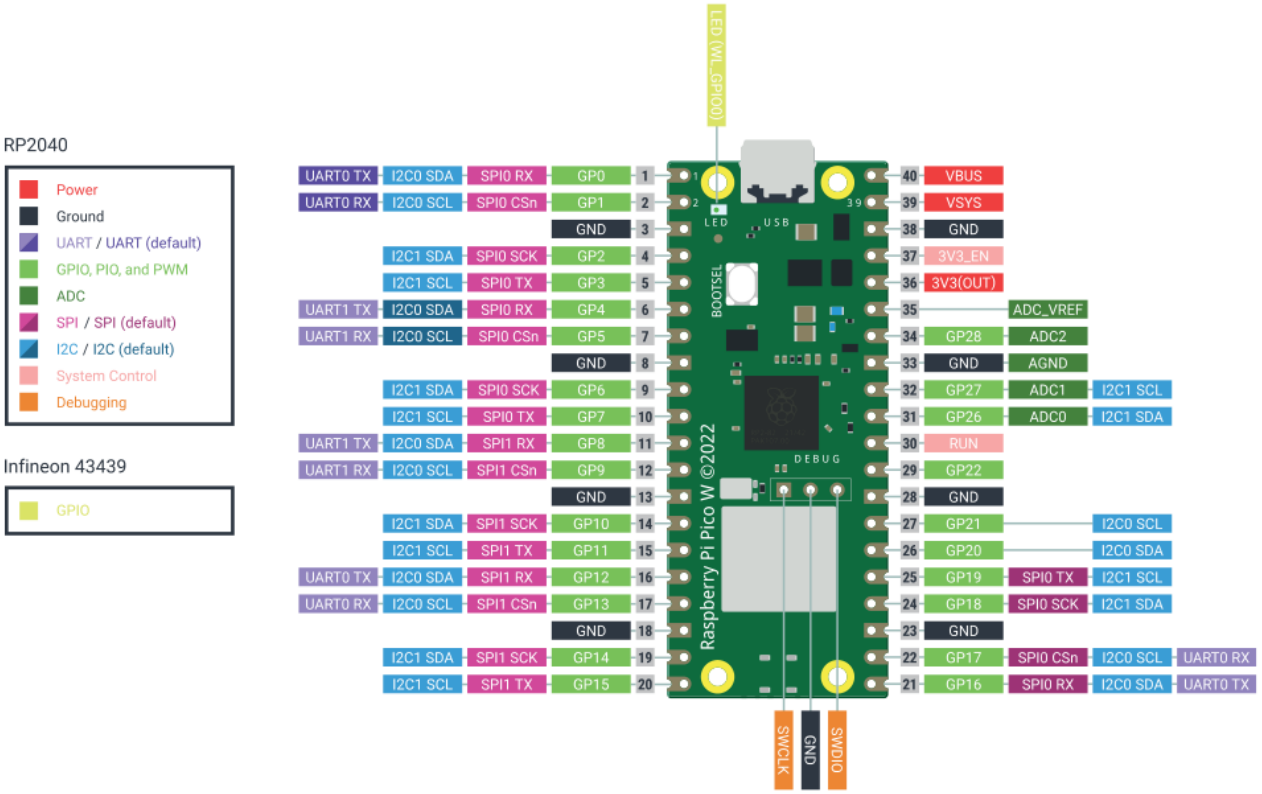
AICTE IDEA LAB
Embedded and AI Robotics

Hands-On Practice: Week 2



Experiment 1: Blinking Onboard LED on Pico W

```
#Code
from machine import Pin
import time

led = Pin("LED",Pin.OUT) # Onboard LED on Pico W is connected to
                         # GPIO 0 of Wireless module
while True:
    led.toggle()
    time.sleep(0.5)      #Delay of 0.5 s
```

Experiment 2: Blinking RGB LED with Pico W

#Code

```
from machine import Pin
import time

bled=Pin(0,Pin.OUT)
gled=Pin(1,Pin.OUT)
rled=Pin(2,Pin.OUT)
while True:

    bled.value(1) #Set led turn ON
    time.sleep(1) #Delay of 1 sec
    bled.value(0) #Set led turn OFF
    time.sleep(1)
    gled.value(1) #Set led turn ON
    time.sleep(1)
    gled.value(0) #Set led turn OFF
    time.sleep(1)
    rled.value(1) #Set led turn ON
    time.sleep(1)
    rled.value(0) #Set led turn OFF
    time.sleep(1)
```

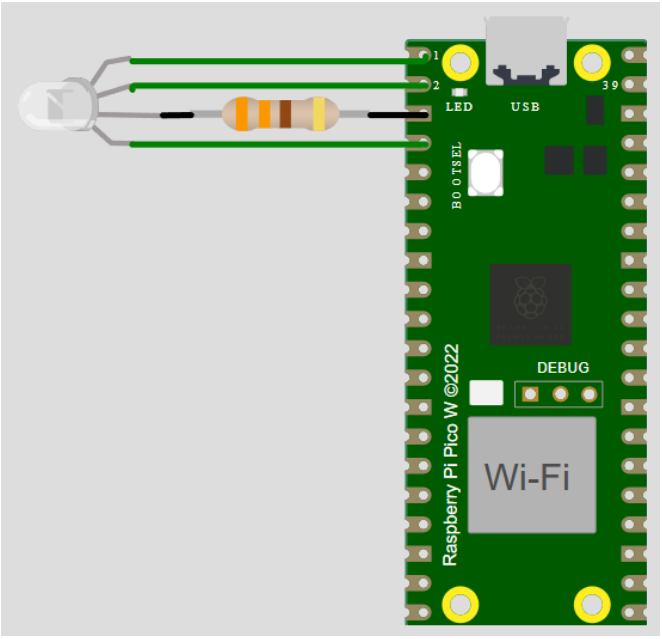


Fig 1: Circuit Diagram for 2 through 4

Experiment 3: Blinking RGB LED with Pico W using PWM

```
from machine import Pin,PWM
import utime
import random
rled = PWM(Pin(2))
gled = PWM(Pin(1))
bled = PWM(Pin(0))
# Define the frequency
rled.freq(2000)
gled.freq(2000)
bled.freq(2000)

while True:
    rled.duty_u16(65535)
    utime.sleep(1)
    rled.duty_u16(0)
    utime.sleep(1)
    gled.duty_u16(65535)
    utime.sleep(1)
    gled.duty_u16(0)
    utime.sleep(1)
    bled.duty_u16(65535)
    utime.sleep(1)
```

```
    bled.duty_u16(0)
    utime.sleep(3)
```

Experiment 4: Generate colours randomly using RGB LED with Pico W

```
from machine import Pin,PWM
import utime
import random
rled = PWM(Pin(2))
gled = PWM(Pin(1))
bled = PWM(Pin(0))
# Define the frequency
rled.freq(2000)
gled.freq(2000)
bled.freq(2000)

while True:
    # range of random numbers
    R=random.randint(0,65535)
    G=random.randint(0,65535)
    B=random.randint(0,65535)
    print(R,G,B)
    rled.duty_u16(R)
    gled.duty_u16(G)
    bled.duty_u16(B)
    utime.sleep(3)
```

For colours of choice, values of R,G,B can be set individually

Experiment 5: Generate colours randomly on button press using RGB LED with Pico W

```
#Generating Random Colours on button press
from machine import Pin,PWM
import utime
import random
button = Pin(4, Pin.IN, Pin.PULL_UP)
rled = PWM(Pin(2))
gled = PWM(Pin(1))
bled = PWM(Pin(0))
# Define the frequency
rled.freq(1000)
gled.freq(1000)
bled.freq(1000)

def set_color(r, g, b):
    rled.duty_u16(r)
    gled.duty_u16(g)
    bled.duty_u16(b)

# Initialize LED to 'Off'
set_color(0, 0, 0)
```

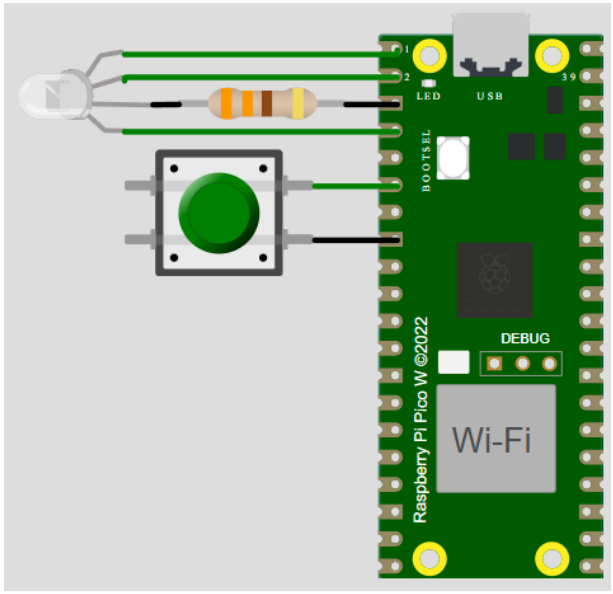


Fig 2: Circuit Diagram for 5 & 6

```
while True:
    # Check if button is pressed (LOW due to PULL_UP)
    if button.value() == 0:
        time.sleep(0.05)  # Debounce delay
        if button.value() == 0:

            # Generate random 16-bit values (0 to 65535) for each color channel
            R = random.randint(0, 65535)
            G = random.randint(0, 65535)
            B = random.randint(0, 65535)

            print(f"Instant Color -> R: {R}, G: {G}, B: {B}")

            # Directly update the LED state without any transition loop
            set_color(R, G, B)

            # Wait for button release to prevent accidental multi-clicks
            while button.value() == 0:
                time.sleep(0.05)

    time.sleep(0.01)
```

Experiment 6: Cycle through different colours on button press using RGB LED with Pico W

#Code

```
#Generating Random Colours
from machine import Pin,PWM
import time
import random
button = Pin(4, Pin.IN, Pin.PULL_UP)
rled = PWM(Pin(2))
gled = PWM(Pin(1))
bled = PWM(Pin(0))
# Define the frequency
rled.freq(1000)
gled.freq(1000)
bled.freq(1000)

# Define custom colors using 16-bit duty cycles (0 to 65535)
# This allows for rich, customized mixed colors like orange and pink
colors = [
    (65535, 0, 0),      # Bright Red
    (0, 65535, 0),      # Bright Green
    (0, 0, 65535),      # Bright Blue
    (65535, 15000, 0),  # Orange
    (65535, 0, 20000),  # Pink / Hot Magenta
    (0, 65535, 30000),  # Teal / Turquoise
    (15000, 0, 40000),  # Purple
    (0, 0, 0)           # Off
]

color_index = 0

def set_color(r, g, b):
    # Set the duty cycle for each color channel
    rled.duty_u16(r)
    gled.duty_u16(g)
    bled.duty_u16(b)

# Initialize LED to 'Off'
set_color(0, 0, 0)

while True:
    # Check if button is pressed (LOW due to PULL_UP)
    if button.value() == 0:
        time.sleep_ms(50)  # Debounce delay
        if button.value() == 0:
            # Move to next color profile
            color_index = (color_index + 1) % len(colors)
```

```
            set_color(*colors[color_index])
            print(f"Switched to color profile index {color_index}")

            # Wait for button release to prevent skipping colors
            while button.value() == 0:
                time.sleep(0.05)

    time.sleep(0.01)
```

Experiment 7: Switch buzzer on and off using Pico W

#Code

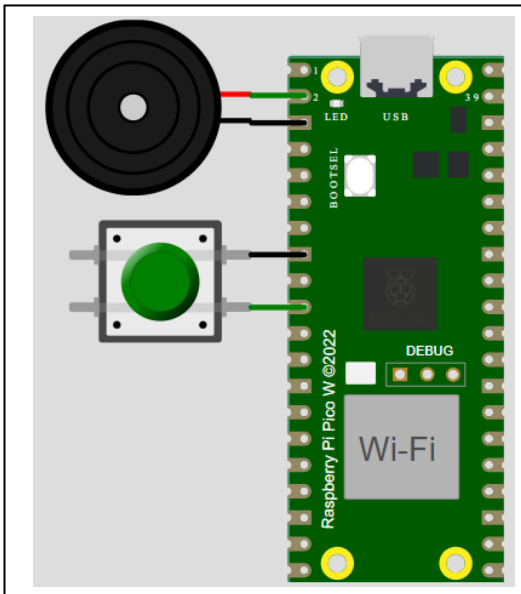


Fig 3: Circuit diagram for 7 & 8

```
import machine
import utime

#Buzzer on and off
# Configure GPIO 1 as an output for the buzzer
buzzer = machine.Pin(1, machine.Pin.OUT)

# Configure GPIO 7 as an input
# with an internal pull-up resistor
button = machine.Pin(7, machine.Pin.IN,
machine.Pin.PULL_UP)

while True:
    # button.value() is 0 when pressed
    #(due to pull-up)
    if button.value() == 0:
        buzzer.value(1)  # Turn buzzer on
    else:
        buzzer.value(0)  # Turn buzzer off

    utime.sleep(0.05) # Small delay to prevent bouncing
```

Experiment 8: Play different tones on buzzer on button press using Pico W

```
# Set up GP1 as a PWM pin for audio generation
buzzer = machine.PWM(machine.Pin(1))

# Set up GP7 as a button with an internal pull-up resistor
button = machine.Pin(7, machine.Pin.IN, machine.Pin.PULL_UP)

# Define musical frequencies (in Hz)
TONE_LOW = 262    # C4 note
TONE_MED = 440    # A4 note
TONE_HIGH = 880   # A5 note
```

```
# Duty cycle controls volume/power (32768 is 50% duty cycle, optimal for buzzers)
VOLUME_ON = 32768
VOLUME_OFF = 0

while True:
    if button.value() == 0:  # Button is pressed
        # Play Low Tone
        buzzer.freq(TONE_LOW)
        buzzer.duty_u16(VOLUME_ON)
        utime.sleep(0.3)

        # Play Medium Tone
        buzzer.freq(TONE_MED)
        buzzer.duty_u16(VOLUME_ON)
        utime.sleep(0.3)

        # Play High Tone
        buzzer.freq(TONE_HIGH)
        buzzer.duty_u16(VOLUME_ON)
        utime.sleep(0.3)

        # Turn off buzzer after the sequence finishes
        buzzer.duty_u16(VOLUME_OFF)

        # Wait until button is released to prevent looping the sound immediately
        while button.value() == 0:
            utime.sleep(0.05)

    utime.sleep(0.05)
```

Experiment 9: Interfacing Analog joystick with Pico W

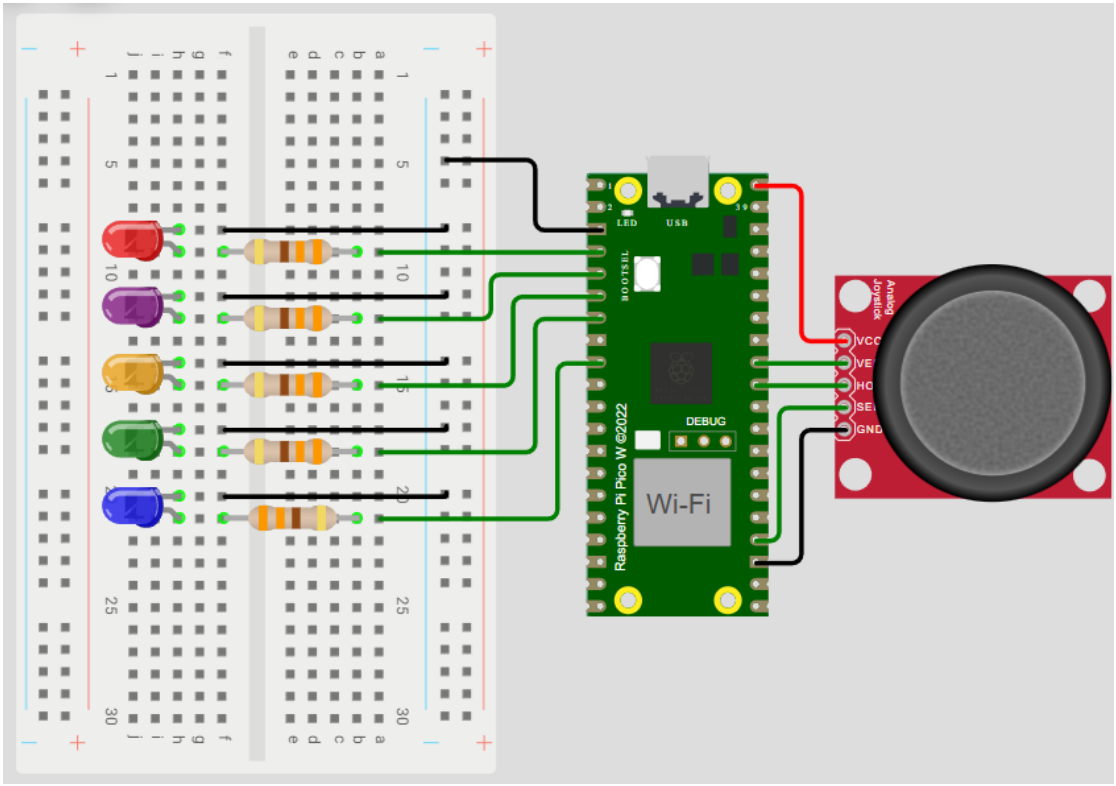


Fig 4: Circuit Diagram for 9

#Code

```
import time
from machine import Pin, ADC

xAxis = ADC(Pin(27))
yAxis = ADC(Pin(26))
sel = Pin(18, Pin.IN, Pin.PULL_UP)

led_up = Pin(2, Pin.OUT)
led_down = Pin(3, Pin.OUT)
led_middle = Pin(4, Pin.OUT)
led_left = Pin(5, Pin.OUT)
led_right = Pin(6, Pin.OUT)

while True:
    xValue = xAxis.read_u16()
    yValue = yAxis.read_u16()
    buttonValue = sel.value()
    xStatus = "middle"
    yStatus = "middle"
    buttonStatus = "not pressed"
```

```
    led_left.value(0)
    led_down.value(0)
    led_up.value(0)
    led_right.value(0)
    led_middle.value(0)

# Check the x and y Value to determine the status of the joystick
    if buttonValue == 0:
        buttonStatus = "pressed"
        led_middle.value(1)

    if xValue <= 600:
        xStatus = "left"
        led_left.value(1)
        led_middle.value(0)

    if xValue >= 60000:
        xStatus = "right"
        led_right.value(1)
        led_middle.value(0)

    if yValue <= 600:
        yStatus = "up"
        led_up.value(1)
        led_middle.value(0)

    if yValue >= 60000:
        yStatus = "down"
        led_down.value(1)
        led_middle.value(0)

    time.sleep(0.2)
```